

1. Universal service investigation commands

```
systemctl status SERVICE --no-pager systemctl cat SERVICE systemctl show SERVICE --no-pager systemctl show SERVICE -p
FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec systemctl is-enabled SERVICE
systemctl is-active SERVICE systemctl list-dependencies SERVICE systemctl list-dependencies --reverse SERVICE
journalctl -u SERVICE -b --no-pager
```

2. Locate the exact service file

```
systemctl cat SERVICE systemctl show SERVICE -p FragmentPath systemctl show SERVICE -p DropInPaths
```

Use systemctl cat instead of guessing whether the unit lives under /etc/systemd/system, /lib/systemd/system or /usr/lib/systemd/system.

3. Find the real program and PID

```
systemctl show SERVICE -p ExecStart,MainPID PID=$(systemctl show -p MainPID --value SERVICE); ps -fp "$PID"
PID=$(systemctl show -p MainPID --value SERVICE); readlink -f /proc/$PID/exe
```

4. Ports, devices and files

```
sudo ss -ltnp sudo lsof -nP -iTCP -sTCP:LISTEN sudo lsof -p $(systemctl show -p MainPID --value SERVICE) 2>/dev/null
ls -l /dev/ttyS* /dev/ttyUSB* /dev/ttyACM* /dev/ttyGS* 2>/dev/null gpioinfo 2>/dev/null ls -l /dev/watchdog*
2>/dev/null
```

5. Your GNSS startup chain

```
systemd → gnss.service → /opt/gnss/start_gnss.sh → /usr/bin/java -jar /home/debian/GNSS_Rover_Base.jar Java → TCP
192.168.10.100:2000 → serial-bridge → /dev/ttyS0 systemd → gnss_gui_server.service → /home/debian/gnss_gui_server →
TCP :8080
```

The JAR is not itself a systemd service. systemd manages the wrapper service.

6. Detailed custom/project service dossiers

gnss.service

Service file: /etc/systemd/system/gnss.service

```
systemctl status gnss.service --no-pager systemctl cat gnss.service systemctl show gnss.service -p
FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u gnss.service -b -n 200
--no-pager
```

Full unit from supplied inventory:

[Unit]

Description=GNSS Java Base Station Auto Start

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

User=debian

WorkingDirectory=/opt/gnss

ExecStart=/opt/gnss/start_gnss.sh

Restart=always

RestartSec=10

SuccessExitStatus=143

[Install]

WantedBy=multi-user.target

```
what actually runs: /opt/gnss/start_gnss.sh → /usr/bin/java -jar /home/debian/GNSS_Rover_Base.jar
```

User/privilege: debian (UID/GID 1000)

Hardware: No direct tty ownership observed. /dev/ttyS0 is owned by serial-bridge.

Ports: Java connects to GNSS serial bridge at 192.168.10.100:2000; remote NTRIP endpoint is application-controlled.

Files/configuration: /home/debian/gnss/config/*; observed ip_port.txt, ip_setting.txt, save_gnss_receiver_settings.txt, version.txt; /opt/gnss working tree.

Dependencies: network-online.target; RequiresMountsFor=/opt/gnss was observed in the service configuration/context.

If stopped: Stops the Java GNSS application; GUI server remains independent.

If crashed: systemd restarts after ~10 seconds because Restart=always.

Safe interaction: Safe via defined TCP/API/file interfaces. Do not open ttyS0 or take TCP 2000.

Leave untouched: YES for unrelated features.

gnss_gui_server.service

Service file: /etc/systemd/system/gnss_gui_server.service

```
systemctl status gnss_gui_server.service --no-pager systemctl cat gnss_gui_server.service systemctl show
gnss_gui_server.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl
-u gnss_gui_server.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=GNSS GUI Server Service

After=network.target

[Service]

Type=simple

ExecStart=/home/debian/gnss_gui_server

WorkingDirectory=/home/debian

Restart=always

RestartSec=5

User=debian

[Install]

WantedBy=multi-user.target

What actually runs: /home/debian/gnss_gui_server

User/privilege: debian

Hardware: No hardware device ownership observed.

Ports: TCP 0.0.0.0:8080.

Files/configuration: ./gnss/ui/; ./gnss/config/; observed update.flag, ipport.txt, password.txt, wifi_setting.txt, ip_port.txt, save_gnss_receiver_settings.txt, .sys_cal.dat.

Dependencies: network.target.

If stopped: Web GUI/API on TCP 8080 becomes unavailable; GNSS Java service remains separate.

If crashed: Restart=always after ~5 seconds.

Safe interaction: Safe as an HTTP/API client if the interface is intended for it. Never bind TCP 8080 or overwrite its files.

Leave untouched: YES for unrelated features.

gnss-watchdog.service

Service file: /etc/systemd/system/gnss-watchdog.service

```
systemctl status gnss-watchdog.service --no-pager systemctl cat gnss-watchdog.service systemctl show
gnss-watchdog.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl
-u gnss-watchdog.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=GNSS Hardware Watchdog Service

After=multi-user.target

```
[Service]
Type=simple
ExecStartPre=/bin/sh -c 'until [ -e /dev/watchdog1 ]; do sleep 1; done'
ExecStart=/opt/gnss/watchdog
User=root
Restart=always
RestartSec=2
```

```
[Install]
WantedBy=multi-user.target
```

What actually runs: /opt/gnss/watchdog

User/privilege: root

Hardware: /dev/watchdog1

Ports: None observed.

Files/configuration: Hardware watchdog device; journal messages such as watchdog kicks.

Dependencies: multi-user.target and device availability.

If stopped: Potentially dangerous: if the watchdog remains armed, board reset may occur when it times out.

If crashed: systemd attempts restart after ~2 seconds.

Safe interaction: Do not access /dev/watchdog1 from another process unless you deliberately redesign watchdog ownership.

Leave untouched: YES; treat as safety-critical.

gnss_gpio.service

Service file: /etc/systemd/system/gnss_gpio.service

```
systemctl status gnss_gpio.service --no-pager systemctl cat gnss_gpio.service systemctl show gnss_gpio.service -p
FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u gnss_gpio.service -b -n
200 --no-pager
```

Full unit from supplied inventory:

```
[Unit]
Description=GNSS GPIO Service
```

```
[Service]
Type=simple
ExecStart=/opt/gnss/gnss_gpio
User=root
Restart=always
RestartSec=2
```

```
[Install]
WantedBy=multi-user.target
```

What actually runs: /opt/gnss/gnss_gpio

User/privilege: root

Hardware: GPIO lines; exact line ownership must be verified with gpioinfo on the live board.

Ports: None observed.

Files/configuration: Application-controlled GPIO/configuration; exact paths must be verified from the executable/source.

Dependencies: Normal boot target.

If stopped: GPIO-controlled hardware behavior may stop or remain in its last state depending on the application.

If crashed: systemd restarts it.

Safe interaction: Do not claim or toggle the same GPIO lines until ownership is mapped.

Leave untouched: YES unless changing GPIO functionality.

serial-bridge.service

Service file: /etc/systemd/system/serial-bridge.service

```
systemctl status serial-bridge.service --no-pager systemctl cat serial-bridge.service systemctl show serial-bridge.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u serial-bridge.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Serial-to-LAN Bridge

[Service]

Type=simple

ExecStart=/opt/gnss/serial_bridge

User=root

Restart=always

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/serial_bridge

User/privilege: root

Hardware: /dev/ttyS0 (observed owner)

Ports: TCP 2000; GNSS Java connects to 192.168.10.100:2000.

Files/configuration: Exact application log/config paths should be verified with the commands in this guide.

Dependencies: Normal boot target; serial device availability.

If stopped: GNSS serial-to-network data path on TCP 2000 stops.

If crashed: systemd restarts it.

Safe interaction: New services should be TCP clients of :2000 rather than opening ttyS0 directly, unless serial-bridge is intentionally redesigned.

Leave untouched: YES; this is a critical resource-owner service.

mcu_server.service

Service file: /etc/systemd/system/mcu_server.service

```
systemctl status mcu_server.service --no-pager systemctl cat mcu_server.service systemctl show mcu_server.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u mcu_server.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=MCU Server

[Service]

Type=simple

ExecStart=/opt/gnss/mcu_server

User=root

Restart=always

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/mcu_server

User/privilege: root

Hardware: /dev/ttyS3 (observed owner)

Ports: TCP 2001.

Files/configuration: Exact application log/config paths should be verified on the live board.

Dependencies: Normal boot target; serial device availability.

If stopped: MCU communication path stops.

If crashed: systemd restarts it.

Safe interaction: Do not open ttyS3 or bind TCP 2001 from another service.

Leave untouched: YES unless changing MCU architecture.

ip-manager.service

Service file: /etc/systemd/system/ip-manager.service

```
systemctl status ip-manager.service --no-pager systemctl cat ip-manager.service systemctl show ip-manager.service -p
FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u ip-manager.service -b
-n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=IP Manager

[Service]

Type=simple

ExecStart=/opt/gnss/ip-manager

User=root

Restart=on-failure

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/ip-manager

User/privilege: root

Hardware: Network interfaces; exact interface list must be verified from logs/config.

Ports: None directly observed.

Files/configuration: Observed GNSS network/IP configuration files under /home/debian/gnss/config.

Dependencies: Network stack.

If stopped: IP configuration/application behavior may stop; inventory shows it can apply settings and restart gnss.service.

If crashed: Restart-on-failure.

Safe interaction: New network service should not independently rewrite the same IP configuration or restart gnss.service unless coordinated.

Leave untouched: YES unless deliberately changing network configuration architecture.

gnss-wifi-setup.service

Service file: /etc/systemd/system/gnss-wifi-setup.service

```
systemctl status gnss-wifi-setup.service --no-pager systemctl cat gnss-wifi-setup.service systemctl show
gnss-wifi-setup.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl
-u gnss-wifi-setup.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=GNSS WiFi Setup

Before=hostapd.service

[Service]

Type=oneshot

ExecStart=/opt/gnss/gnss-wifi-setup

RemainAfterExit=yes

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/gnss-wifi-setup

User/privilege: default/root context; verify with systemctl show

Hardware: Wi-Fi configuration; exact hardware device ownership belongs to hostapd/iwd/network stack.

Ports: No direct listener observed.

Files/configuration: hostapd.conf plus GNSS Wi-Fi configuration files; exact paths should be verified with systemctl cat and journalctl.

Dependencies: Before hostapd.service.

If stopped: Does not behave like a continuously running daemon; it is a configuration/setup action.

If crashed: A oneshot failure can leave AP configuration incomplete.

Safe interaction: Do not overwrite hostapd.conf or related Wi-Fi settings without understanding this setup step.

Leave untouched: YES unless changing AP setup.

gnss-wifi-connect.service

Service file: /etc/systemd/system/gnss-wifi-connect.service

```
systemctl status gnss-wifi-connect.service --no-pager systemctl cat gnss-wifi-connect.service systemctl show
gnss-wifi-connect.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec
journalctl -u gnss-wifi-connect.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Connect BeaglePlay to external WiFi for internet access

[Service]

Type=simple

ExecStart=/opt/gnss/gnss-wifi-connect

Restart=always

RestartSec=10

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/gnss-wifi-connect

User/privilege: Verify with systemctl show; inventory emphasizes behavior rather than assuming root/debian.

Hardware: Wi-Fi interface.

Ports: No listener; outbound Wi-Fi/network connections.

Files/configuration: Wi-Fi configuration/logs as used by the script.

Dependencies: Wi-Fi/network stack.

If stopped: Automatic external Wi-Fi connection attempts stop.

If crashed: Restart loop is possible; journal showed repeated connection errors in the captured state.

Safe interaction: Do not create a second service that fights over the same Wi-Fi interface/profile.

Leave untouched: INVESTIGATE before modifying; journal showed 'invalid extra argument --' errors.

quectel-check.service

Service file: /etc/systemd/system/quectel-check.service

```
systemctl status quectel-check.service --no-pager systemctl cat quectel-check.service systemctl show
quectel-check.service -p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl
-u quectel-check.service -b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Quectel EC200U Auto Connect Check

[Service]

Type=simple

ExecStart=/opt/gnss/quectel-check

Restart=always

[Install]

WantedBy=multi-user.target

What actually runs: /opt/gnss/quectel-check

User/privilege: Verify with systemctl show.

Hardware: Quectel EC200U modem/USB serial interfaces.

Ports: Network path is provided by modem; no local listener observed.

Files/configuration: Modem/network status logs/config; verify live.

Dependencies: Modem and network stack.

If stopped: Automatic modem health/reconnect checks stop.

If crashed: Restart policy attempts recovery.

Safe interaction: Do not create a competing modem reset/reconnect loop.

Leave untouched: YES unless changing EC200U management.

modem_init.service

Service file: /etc/systemd/system/modem_init.service

```
systemctl status modem_init.service --no-pager systemctl cat modem_init.service systemctl show modem_init.service -p
FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u modem_init.service -b
-n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Modem initialization

[Service]

Type=oneshot

ExecStart=/opt/gnss/modem_init

RemainAfterExit=yes

What actually runs: /opt/gnss/modem_init

User/privilege: Verify with systemctl show.

Hardware: Quectel modem interfaces.

Ports: No local listener observed.

Files/configuration: Modem initialization configuration; verify live.

Dependencies: Modem availability.

If stopped: Initialization action itself is not continuous; after completion the modem remains configured.

If crashed: Oneshot failure can leave modem incompletely initialized.

Safe interaction: Do not duplicate AT-command initialization in a new service without coordinating with this unit.

Leave untouched: YES.

modem_status.service

Service file: /etc/systemd/system/modem_status.service

```
systemctl status modem_status.service --no-pager systemctl cat modem_status.service systemctl show modem_status.service
-p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u modem_status.service
-b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Modem status

[Service]

Type=oneshot

ExecStart=/opt/gnss/modem_status

What actually runs: /opt/gnss/modem_status

User/privilege: Verify with systemctl show.

Hardware: Quectel modem interfaces.

Ports: No local listener observed.

Files/configuration: Status output/logs; verify live.

Dependencies: Modem availability.

If stopped: Only the status check stops; it is normally triggered by modem_status.timer.

If crashed: A failed check is visible in journal/timer history.

Safe interaction: Prefer reading its status output/logs instead of running competing modem probes.

Leave untouched: Verify before changing.

net-timesync.service

Service file: /etc/systemd/system/net-timesync.service

```
systemctl status net-timesync.service --no-pager systemctl cat net-timesync.service systemctl show net-timesync.service
-p FragmentPath,DropInPaths,ExecStart,User,Group,WorkingDirectory,Restart,RestartSec journalctl -u net-timesync.service
-b -n 200 --no-pager
```

Full unit from supplied inventory:

[Unit]

Description=Network time synchronization helper

[Service]

Type=oneshot

ExecStart=/opt/gnss/net-timesync

What actually runs: /opt/gnss/net-timesync

User/privilege: Verify with systemctl show.

Hardware: None directly observed.

Ports: Outbound time/network access.

Files/configuration: System clock and helper logs/config as applicable.

Dependencies: Network availability.

If stopped: The synchronization action does not run.

If crashed: Time may remain unsynchronized until another synchronization mechanism succeeds.

Safe interaction: Do not introduce a second competing time-setting service without understanding systemd-timesyncd and this helper.

Leave untouched: YES.

7. Running services — complete reference

8. Failed services

bring-up-eth2.service

Bring up eth2 if available

Failed; likely legacy/custom eth2 bring-up path; investigate before network changes.

```
systemctl status bring-up-eth2.service --full --no-pager systemctl cat bring-up-eth2.service systemctl show
bring-up-eth2.service -p FragmentPath,DropInPaths,ExecStart,User,Group,Restart,RestartSec journalctl -u
bring-up-eth2.service -b -n 200 --no-pager
```

dnsmasq.service

dnsmasq - A lightweight DHCP and caching DNS server

Failed; may relate to AP/DHCP; map against hostapd/ap0 before touching.

```
systemctl status dnsmasq.service --full --no-pager systemctl cat dnsmasq.service systemctl show dnsmasq.service -p
FragmentPath,DropInPaths,ExecStart,User,Group,Restart,RestartSec journalctl -u dnsmasq.service -b -n 200 --no-pager
```

networking.service

Raise network interfaces

Failed traditional ifupdown service; systemd-networkd/NetworkManager are also present.

```
systemctl status networking.service --full --no-pager systemctl cat networking.service systemctl show
networking.service -p FragmentPath,DropInPaths,ExecStart,User,Group,Restart,RestartSec journalctl -u networking.service
-b -n 200 --no-pager
```

9. Network and serial ownership

TCP 8080 = gnss_gui_server; TCP 2000 = serial-bridge; TCP 2001 = mcu_server; TCP 1880 = Node-RED; TCP 80 = nginx; TCP 22 = SSH.

/dev/ttyS0 = serial-bridge; /dev/ttyS3 = mcu_server; /dev/ttyS2 = serial-getty; /dev/ttyGS0 = USB serial getty; /dev/watchdog1 = gnss-watchdog. Map GPIO lines with gpioinfo.

10. Safe new-service procedure

Baseline: `systemctl --failed; systemctl list-units --type=service --state=running; sudo ss -lntup`.

Map resource ownership: ports, UARTs, GPIOs, watchdog, USB serial interfaces and configuration files.

Run the new executable manually before making systemd manage it.

Use the same production user during testing.

Create dedicated `/opt//` directories and configuration/log locations.

Create `/etc/systemd/system/.service`.

```
sudo systemctl daemon-reload.
```

```
sudo systemctl start .service.
```

```
systemctl status .service --full --no-pager.
```

```
journalctl -u .service -b -n 200 --no-pager.
```

Verify ports/devices and then test GNSS Java, GUI, serial bridge and MCU functions.

Only after testing: `sudo systemctl enable .service`.

Document rollback and every changed file.

Reboot only during a controlled test window.

11. Known risks

`gnss-wifi-connect.service` reported repeated Wi-Fi failures and an 'invalid extra argument --' error.

`systemd-networkd` and `NetworkManager` are active while `networking.service` is failed; determine interface ownership before network changes.

`bring-up-eth2.service` is failed although `eth2` had an IP address at capture time.

`dnsmasq.service` is failed while AP/DHCP-related services/interfaces exist; map `hostapd`, `dnsmasq` and `ap0`.

`daily-reboot.timer` description and captured schedule appear inconsistent; verify before changing.

A CRLF-named `gnss-wifi-setup` unit file was visible in `/etc/systemd/system`; investigate before deleting.

TCP 3000 was occupied by a node process; identify its owner before allocating the port.

Map every `/dev/ttyUSB*` before implementing modem features because multiple modem helpers may issue AT commands.

12. Final engineering rules

One owner per UART.

One owner per watchdog.

One clear owner per GPIO line.

One listener per TCP/UDP port.

One authoritative writer per configuration file.

Do not add a second network manager for an interface already managed elsewhere.

Prefer APIs/TCP/IPC over opening another application's hardware device directly.

Keep independent features in independent services.

Use systemd dependencies only for real dependencies.

Test manually first, then under systemd.

Capture a baseline before and after changes.

Do not 'clean up' failed services without proving they are obsolete.

Never stop watchdog, SSH, network or OTA services casually on a remote board.

13. Evidence boundary

This document is based on the supplied board inventory and prior service guide. For package services whose exact unit text was not supplied, it deliberately provides the exact command to retrieve the authoritative board-specific unit instead of fabricating it.

Document use: For any service, first run `systemctl cat SERVICE` and copy the complete unit. Then inspect the process, user, ports, devices, files and journal. Do not modify an existing service until its resource ownership is understood.